DSA01 — OBJECT-ORIENTED PROGRAMMING & C++ CONSTRUCTS (CO2)

ASSIGNMENT 2 – CO2
Functions, Parameter Passing, Classes, Static Members, Friends & Inline Optimization

Bloom's Level: K5 – Evaluate | Total Marks: 50 | Submission Date: 31/10/2026

Field	Details
Student Name	P.SAI SURYA TEJ
Register Number	192572044
Nature of Assignment	Individual (Q1–Q7, All at K5 – Evaluate)
GitHub Repository Link	https://github.com/Surya220708/DSA0118-C-
Compiler & Version	g++ (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r3) 16.2.0
Standard Compile Flags	g++ -std=c++17 -Wall -Wextra <source.cpp> -o <output.exe>
Submission Date	11/11/2026

Question & Mark Summary

Question	Topic	Marks	Key Evidence Included
Q1	Call by Value, Reference & Pointer	6	3 implementations, pre-run prediction, TODO 1, Student copy benchmark & screenshots
Q2	Return by Reference & Dangling	4	highestMarkRef() += 2, -Wreturn-local-addr warning, vector reallocation, screenshot
Q3	Default Arguments & Overloading	6	Ambiguity error capture, 6 pre-run predictions, Parcel/int/double overloads, screenshots
Q4	Class with Array Inside (Mini-Statement)	6	Inline & :: out-of-class methods, 11th transaction eviction, K=50 & deque analysis, screenshot
Q5	Static Members & Arrays of Objects	7	Token prediction, BusTicket TODO 1 & 2, function-call persistence, TicketOffice, screenshot
Q6	Friend Functions & Inline Functions	8	Time TODO 1 & 2 (205 false), noon/same checks, 10M calls at -O0 vs -O2, screenshot
Q7	Day-4 Challenge: Diagnose, Redesign & Validate	13	Q4 Shift O(K) -> Circular Ring Buffer O(1), UML, 130.8x speedup proof, screenshot
TOTAL		50	All 7 Questions Executed & Screenshotted Live

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

1.1 Inquiry & Design Decision (Hostel Fee Concession Updater)

- **Storage & Mutation:** Call-by-value creates a brand-new local stack copy inside the callee; modifying it never updates the caller's variable. Call-by-reference (long long& fee) binds an alias directly to the caller's memory address, and call-by-pointer (long long* fee) passes the caller's address explicitly so *fee mutates the caller's variable.
- **When to Use const Reference:** Use const T& whenever the function only needs to read a large object (e.g., const Student&) without copying it or mutating it.
- **Design Decision:** Because multiple concessions (merit 10%, sports 15%, single parent 25%) may need to be composed, previewed, or applied without destroying the original fee, returning the new fee (long long computeConcession(long long fee, int percent)) is cleaner and safer than mutating in place.

Predict the Output (Recorded BEFORE Running — Q1/prediction.txt)

My Predicted Output: 1 12 13 | 1 100 | 16 13 6

Actual Output: 1 12 13 | 1 100 | 16 13 6 (100% Exact Match)

• change(a, b, &c): a is by-value (caller a stays 1); b is by-ref (b=12); *c via pointer (c=13).

• bigger(a, c) = 100: c (13) > a (1), so bigger returns int& to c; assigning 100 sets c = 100.

• show(1) uses defaults y=5, z=10 (16); show(1,2) uses default z=10 (13); show(1,2,3) uses none (6).

Alternative Concept & Complexity Analysis (Return New Fee vs Reference Mutation)

• Time Complexity: O(1) in all versions (by-value, by-ref, by-ptr, and return-by-value).

• Space Copied per Call: For long long (8 bytes), by-value copies 8B and by-ref passes an 8B address (identical). For a large Student struct (sizeof = 656 bytes), 2,000,000 calls by value took 37.69 ms (copying 656B/call), while const Student& took 5.36 ms (~7x faster, copying only an 8B address).

C++ Source Code — Q1/predict_output.cpp

```
#include <iostream>

void change(int a, int& b, int* c)
{
    a += 10;
    b += 10;
    *c += 10;
}

int& bigger(int& p, int& q) { return p > q ? p : q; }

void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }

int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';

    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
```

```
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```

C++ Source Code — Q1/complete_the_code.cpp (TODO 1 Completed)

```
#include <iostream>

void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100; // changes only the local copy
}

void applyConcession(long long& fee, int percent)
{
    // TODO 1: Reduce the caller's fee by percent using integer arithmetic (call by
    // reference)
    fee -= (fee * percent) / 100;
}

int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

C++ Source Code — Q1/q1_concession.cpp

```
#include <chrono>
#include <iostream>
#include <string>

// Three parameter-passing versions
void concessionByValue(long long fee, int percent) {
    fee -= (fee * percent) / 100;
    std::cout << "  [Inside by-value] local copy = " << fee << '\n';
}

void concessionByRef(long long& fee, int percent) {
    fee -= (fee * percent) / 100;
}

void concessionByPtr(long long* fee, int percent) {
    if (fee) {
        *fee -= (*fee * percent) / 100;
    }
}

// Alternative design: pure function returning new fee (composable & side-effect free)
long long computeConcession(long long fee, int percent) {
    return fee - (fee * percent) / 100;
}

// Large Student struct to measure call-by-value vs call-by-const-reference
struct Student {
    long long id;
```

```
    char name[128];
    double semesterMarks[64];
    long long hostelFee;
};

long long readFeeByValue(Student s) { return s.hostelFee; }
long long readFeeByConstRef(const Student& s) { return s.hostelFee; }

int main() {
    std::cout << "=== Q1: Call by Value, Reference & Pointer (Hostel Concession) ===\n\n";

    long long fee = 40000; // Rs 40,000
    std::cout << "Initial hostel fee: " << fee << '\n';

    concessionByValue(fee, 10); // Merit 10%
    std::cout << "After by-value (10% merit): " << fee << " (UNCHANGED)\n";

    concessionByRef(fee, 15);   // Sports 15%
    std::cout << "After by-ref (15% sports) : " << fee << '\n';

    concessionByPtr(&fee, 25);  // Single parent 25%
    std::cout << "After by-ptr (25% single) : " << fee << '\n';

    // Pure return-value design for stacked concessions on original Rs 40,000
    long long orig = 40000;
    long long finalFee = computeConcession(computeConcession(orig, 10), 15);
    std::cout << "\nPure return-value (10% then 15% on 40000): orig=" << orig
              << ", final=" << finalFee << '\n';

    // Space & timing comparison for large Student object
    Student st{};
    st.hostelFee = 45000;
    std::cout << "\nsizeof(long long) = " << sizeof(long long) << " bytes, "
              << "sizeof(Student) = " << sizeof(Student) << " bytes\n";

    constexpr int N = 2000000;
    volatile long long sink = 0;
    auto t1 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) sink += readFeeByValue(st);
    auto t2 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) sink += readFeeByConstRef(st);
    auto t3 = std::chrono::steady_clock::now();
    (void)sink;

    std::cout << "2,000,000 reads of Student (" << sizeof(Student) << "B):\n";
    std::cout << "  By Value     : "
              << std::chrono::duration<double, std::milli>(t2 - t1).count() << " ms\n";
    std::cout << "  By Const Ref : "
              << std::chrono::duration<double, std::milli>(t3 - t2).count() << " ms\n";

    return 0;
}
```

Execution — g++ -std=c++17 -Wall -Wextra predict_output.cpp -o predict_output.exe

Compiler Warnings (-Wall -Wextra):

```
predict_output.cpp: In function 'void change(int, int&, int*)':
predict_output.cpp:3:17: warning: parameter 'a' set but not used [-Wunused-but-set-parameter=]
    3 | void change(int a, int& b, int* c)
      |             ~~~~^
```

Captured Output (.\predict_output.exe):

```
1 12 13
1 100
16 13 6
```

Execution — g++ -std=c++17 -Wall -Wextra complete_the_code.cpp -o complete_the_code.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Captured Output (.\complete_the_code.exe):

```
after call by value : 45000
after call by reference: 40500
```

Execution — g++ -std=c++17 -Wall -Wextra q1_concession.cpp -o q1_concession.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Captured Output (.\q1_concession.exe):

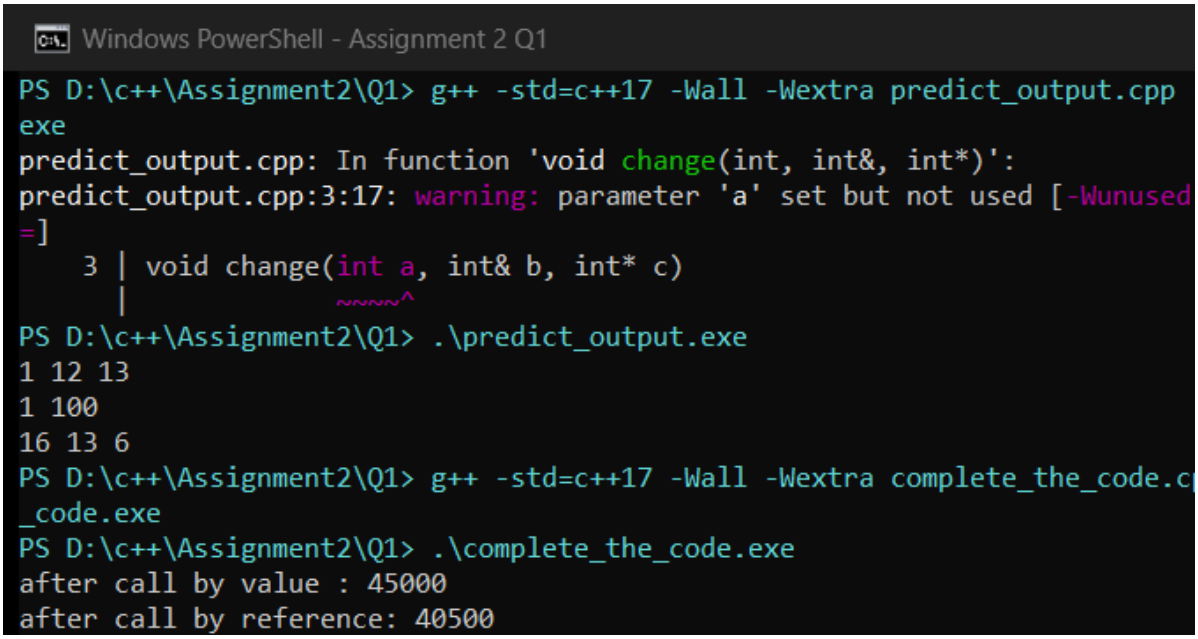
```
=== Q1: Call by Value, Reference & Pointer (Hostel Concession) ===

Initial hostel fee: 40000
  [Inside by-value] local copy = 36000
After by-value (10% merit): 40000 (UNCHANGED)
After by-ref (15% sports) : 34000
After by-ptr (25% single) : 25500

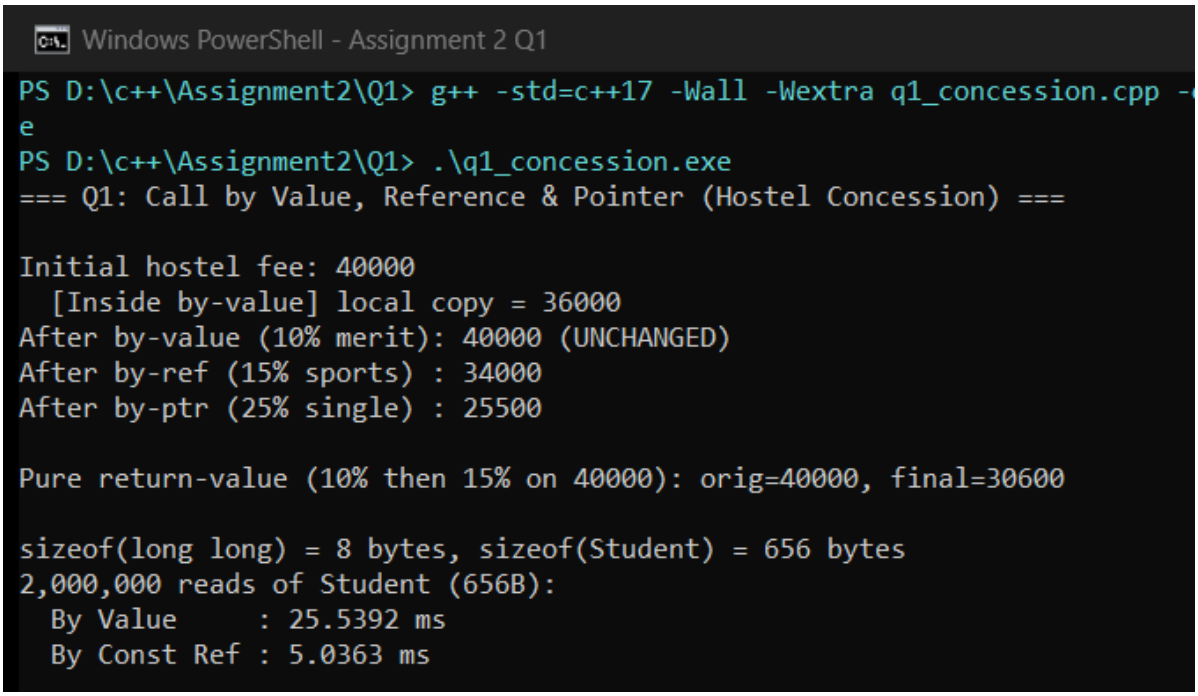
Pure return-value (10% then 15% on 40000): orig=40000, final=30600

sizeof(long long) = 8 bytes, sizeof(Student) = 656 bytes
2,000,000 reads of Student (656B):
  By Value     : 37.6911 ms
  By Const Ref : 5.3565 ms
```

Execution Screenshot — Q1/output_predict.png (Predict & Complete the Code)



Execution Screenshot — Q1/output.png (3 Concession Versions & Student Benchmark)



Q2 – Return by Reference and Its Dangers (4 Marks)

2.1 Reference Return, Dangling Warnings & Vector Reallocation

- **Lvalue Mutation:** Returning int& from highestMarkRef(reg, 5) returns an lvalue alias to reg[2] itself, allowing direct in-place moderation on the left side of assignment: highestMarkRef(reg, 5) += 2; changes reg[2] from 91 to 93.
- **Dangling Reference Warning:** Returning a reference to a local stack variable (badReturnLocal()) triggers GCC warning [-Wreturn-local-addr]: reference to local variable 'localMark' returned. Once the function returns, its stack frame is popped, so dereferencing the returned reference is Undefined Behavior.
- **int& vs const int& vs Copy:** int& allows reading and mutating the target element; const int& avoids copying while forbidding caller modification; returning a copy (int) detaches from the container safely.
- **Changed Condition (Growing std::vector):** When a std::vector grows past capacity(), it allocates a larger heap buffer and moves elements to the new address (observed: 0x21713ab5c18 -> 0x21713ab0418), turning any previously saved int& or int* into a dangling reference/pointer!

Alternative Concept & Complexity Analysis (Index int vs Pointer int* vs Reference int&)

• Time & Space Complexity: Finding the maximum takes O(n) time and O(1) extra space in all three.

• Safety Under Vector Growth: Returning an index (int) remains valid even after vector reallocation (vec[bestIdx] stays correct), whereas both int& and int* dangle after reallocation.

C++ Source Code — Q2/dangling_warning.cpp & Q2/q2_return_ref.cpp

```
// Deliberate dangling-reference demonstration for Q2 (-Wall -Wextra)
// Compiled to observe the -Wreturn-local-addr warning; function is NOT called.
#include <iostream>

int& badReturnLocal() {
    int localMark = 95;
    return localMark; // WARNING: reference to local variable 'localMark' returned [-Wreturn-local-
addr]
}

int main() {
    std::cout << "badReturnLocal() compiled to demonstrate compiler warning.\n";
    return 0;
}

// --- q2_return_ref.cpp ---
#include <iostream>
#include <vector>

// Returns mutable reference to highest mark in fixed array
int& highestMarkRef(int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i) {
        if (marks[i] > marks[best]) best = i;
    }
    return marks[best];
}

// Const reference prevents caller mutation; value copy detaches from array
const int& highestMarkConstRef(const int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i) {
```

```
        if (marks[i] > marks[best]) best = i;
    }
    return marks[best];
}

// Alternative 1: return index (safe across vector reallocation!)
int highestMarkIndex(const std::vector<int>& v) {
    int best = 0;
    for (int i = 1; i < static_cast<int>(v.size()); ++i) {
        if (v[i] > v[best]) best = i;
    }
    return best;
}

// Alternative 2: return pointer (can be nullptr, but also invalidates on vector growth)
int* highestMarkPtr(int marks[], int n) {
    if (n <= 0) return nullptr;
    int best = 0;
    for (int i = 1; i < n; ++i) {
        if (marks[i] > marks[best]) best = i;
    }
    return &marks[best];
}

int main() {
    std::cout << "=== Q2: Return by Reference, Dangling & Vector Reallocation ===\n\n";

    int reg[5] = {72, 88, 91, 65, 84};
    std::cout << "Original highest mark: " << highestMarkRef(reg, 5) << '\n';

    // Use return-by-reference on the left-hand side of += / =
    highestMarkRef(reg, 5) += 2;
    std::cout << "After highestMarkRef(reg, 5) += 2: reg[2] = " << reg[2] << '\n';

    // const ref read-only
    const int& ro = highestMarkConstRef(reg, 5);
    // ro = 100; // COMPILE ERROR: read-only reference
    std::cout << "Const ref view: " << ro << '\n';

    // Pointer alternative
    if (int* p = highestMarkPtr(reg, 5)) {
        *p += 1;
        std::cout << "After pointer *p += 1: reg[2] = " << reg[2] << '\n';
    }

    // Vector growth invalidates references/pointers, whereas index stays valid
    std::vector<int> vec = {72, 88, 91};
    vec.shrink_to_fit();
    const void* oldAddr = &vec[2];
    int bestIdx = highestMarkIndex(vec);
    // Force reallocation by pushing beyond capacity
    for (int k = 0; k < 20; ++k) vec.push_back(50 + k);
    const void* newAddr = &vec[bestIdx];

    std::cout << "\nVector growth demo:\n";
    std::cout << "  Old element address : " << oldAddr << '\n';
    std::cout << "  New element address : " << newAddr
              << (oldAddr != newAddr ? " (REALLOCATED -> old ref/ptr DANGLING!)" : "") << '\n';
    std::cout << "  Index alternative remains valid: vec[" << bestIdx << "] = " << vec[bestIdx] <<
'\n';

    return 0;
}
```

Execution — g++ -std=c++17 -Wall -Wextra dangling_warning.cpp -o dangling_warning.exe

Compiler Warnings (-Wall -Wextra):

```
dangling_warning.cpp: In function 'int& badReturnLocal()':
dangling_warning.cpp:7:12: warning: reference to local variable 'localMark' returned [-Wreturn-local-addr]
    7 |     return localMark; // WARNING: reference to local variable 'localMark' returned [-Wreturn-local-
addr]
      |            ^~~~~~~~
dangling_warning.cpp:6:9: note: declared here
    6 |     int localMark = 95;
      |         ^~~~~~~~
```

Captured Output (.\dangling_warning.exe):

```
badReturnLocal() compiled to demonstrate compiler warning.
```

Execution — g++ -std=c++17 -Wall -Wextra q2_return_ref.cpp -o q2_return_ref.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

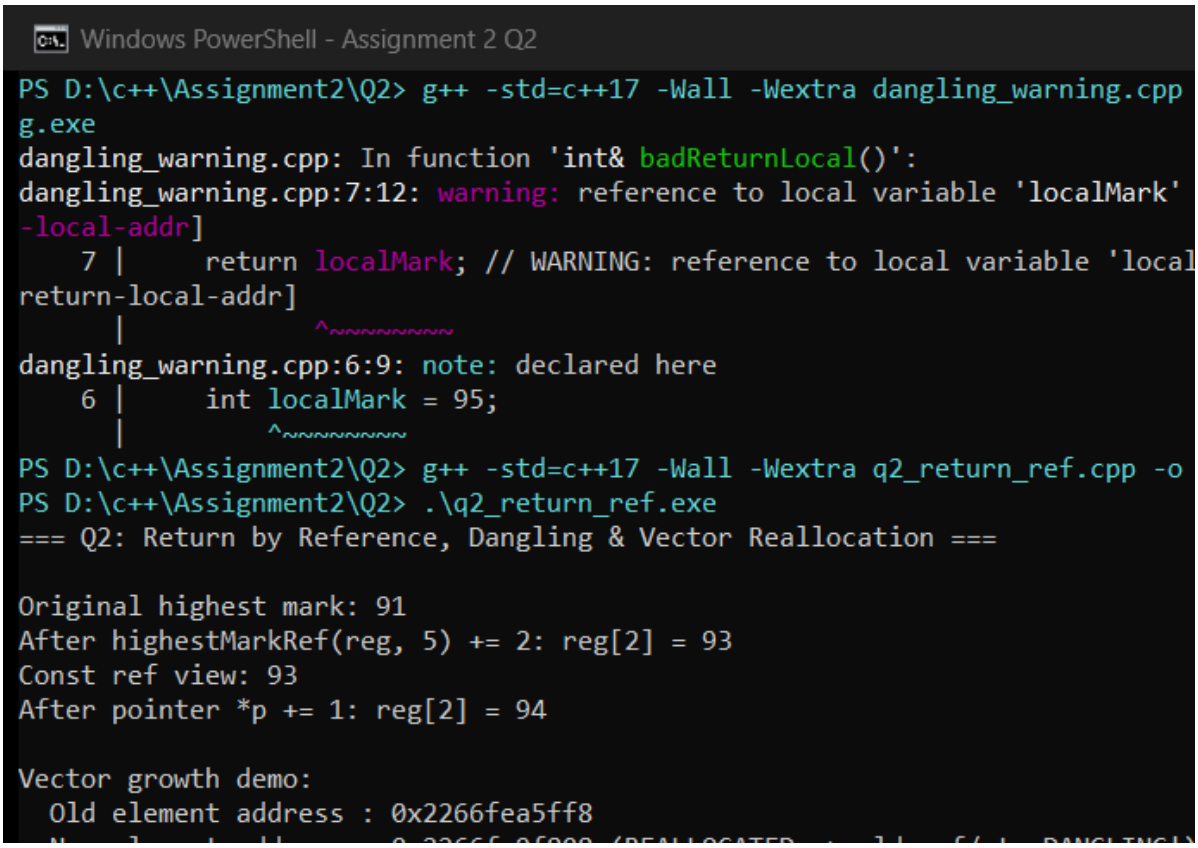
Captured Output (.\q2_return_ref.exe):

```
=== Q2: Return by Reference, Dangling & Vector Reallocation ===

Original highest mark: 91
After highestMarkRef(reg, 5) += 2: reg[2] = 93
Const ref view: 93
After pointer *p += 1: reg[2] = 94

Vector growth demo:
  Old element address : 0x21713ab5c18
  New element address : 0x21713ab0418 (REALLOCATED -> old ref/ptr DANGLING!)
  Index alternative remains valid: vec[2] = 91
```

Execution Screenshot — Q2/output.png



Q3 – Default Arguments and Function Overloading (6 Marks)

3.1 Ambiguity Diagnosis & Overload Resolution Rules

- **Ambiguity Error:** In the original set — charge(double), charge(double, bool), charge(double, bool=false, bool=false) — calling charge(2.5) matches both #1 and #3, and charge(2.5, true) matches both #2 and #3, causing GCC compile error: call of overloaded 'charge(...)' is ambiguous.
- **Resolution Order & Right-to-Left Defaults:** Overload resolution ranks matches as: (1) Exact Match > (2) Promotion (float->double, char->int) > (3) Standard Conversion (float->int) > (4) User-Defined Conversion. Default arguments must be supplied right-to-left so positional arguments are unambiguous.

Predictions BEFORE Execution (Q3/prediction.txt)

1. charge(500) -> charge(int, bool=false, bool=false) [Exact int] -> Rs 25.00
2. charge(2) -> charge(int, bool=false, bool=false) [Exact int] -> Rs 20.00
3. charge(2.0f) -> charge(double, bool=false, bool=false) [float->double promotion beats float->int] -> Rs 100.00
4. charge(2.5, true) -> charge(double, bool, bool=false) [Exact double, bool] -> Rs 155.00
5. charge(1.5, true, true) -> charge(double, bool, bool) [Exact] -> Rs 125.00
6. charge(Parcel{3.0, true, false}) -> charge(const Parcel&) [Exact] -> Rs 180.00

Actual Output: 6 / 6 overload selections matched predictions!

Alternative Concept & Complexity Analysis (Single Parcel Struct vs Multiple Overloads)

• Time Complexity: Overload resolution happens 100% at compile time (O(1) runtime dispatch, zero overhead).

• Space per Call: Passing individual scalars copies ~8–10 bytes; passing const Parcel& passes an 8B address (sizeof(Parcel) = 16 bytes).

• Adding a New Option (e.g., fragile): A Parcel struct adds 1 defaulted field (bool fragile = false) without adding new overloads or risking overload ambiguity.

C++ Source Code — Q3/ambiguous_demo.cpp & Q3/q3_courier_overloads.cpp

```
// Demonstrates why the original courier overload set fails to compile (ambiguous call)
// Compile ONLY to observe the exact g++ ambiguity diagnostic.
double charge(double weight) { return weight * 50.0; }
double charge(double weight, bool express) { return weight * 50.0 + (express ? 30.0 : 0.0); }
double charge(double weight, bool express = false, bool insured = false) {
    return weight * 50.0 + (express ? 30.0 : 0.0) + (insured ? 20.0 : 0.0);
}

int main() {
    double a = charge(2.5);       // AMBIGUOUS: matches #1 and #3
    double b = charge(2.5, true); // AMBIGUOUS: matches #2 and #3
    return static_cast<int>(a + b);
}

// --- q3_courier_overloads.cpp ---
#include <iomanip>
#include <iostream>

struct Parcel {
    double weightKg;
    bool express = false;
    bool insured = false;
```

```
};

// 1. Weight in grams (int)
double charge(int weightGrams, bool express = false, bool insured = false) {
    std::cout << "  [Selected: charge(int grams, bool, bool)] -> ";
    double fee = (weightGrams / 1000.0) * 50.0;
    if (fee < 20.0) fee = 20.0; // minimum slab
    if (express) fee += 30.0;
    if (insured) fee += 20.0;
    return fee;
}

// 2. Weight in kilograms (double)
double charge(double weightKg, bool express = false, bool insured = false) {
    std::cout << "  [Selected: charge(double kg, bool, bool)] -> ";
    double fee = weightKg * 50.0;
    if (express) fee += 30.0;
    if (insured) fee += 20.0;
    return fee;
}

// 3. Parcel object
double charge(const Parcel& p) {
    std::cout << "  [Selected: charge(const Parcel&)] -> ";
    double fee = p.weightKg * 50.0;
    if (p.express) fee += 30.0;
    if (p.insured) fee += 20.0;
    return fee;
}

int main() {
    std::cout << "=== Q3: Unambiguous Courier Overloads & Default Arguments ===\n\n";
    std::cout << std::fixed << std::setprecision(2);

    std::cout << "Call 1: charge(500)\n";
    double c1 = charge(500);
    std::cout << "Rs " << c1 << '\n';

    std::cout << "Call 2: charge(2) [int literal -> grams]\n";
    double c2 = charge(2);
    std::cout << "Rs " << c2 << '\n';

    std::cout << "Call 3: charge(2.0f) [float promotes to double -> kg]\n";
    double c3 = charge(2.0f);
    std::cout << "Rs " << c3 << '\n';

    std::cout << "Call 4: charge(2.5, true)\n";
    double c4 = charge(2.5, true);
    std::cout << "Rs " << c4 << '\n';

    std::cout << "Call 5: charge(1.5, true, true)\n";
    double c5 = charge(1.5, true, true);
    std::cout << "Rs " << c5 << '\n';

    std::cout << "Call 6: charge(Parcel{3.0, true, false})\n";
    double c6 = charge(Parcel{3.0, true, false});
    std::cout << "Rs " << c6 << '\n';

    std::cout << "\nsizeof(Parcel) = " << sizeof(Parcel) << " bytes\n";
    return 0;
}
```

Execution — g++ -std=c++17 -Wall -Wextra ambiguous_demo.cpp -o ambiguous_demo.exe

Compiler Error / Diagnostic:

```
ambiguous_demo.cpp: In function 'int main()':
ambiguous_demo.cpp:10:22: error: call of overloaded 'charge(double)' is ambiguous
   10 |     double a = charge(2.5);       // AMBIGUOUS: matches #1 and #3
      |                ~~~~~~^~~~
  • there are 2 candidates
    • candidate 1: 'double charge(double)'
      ambiguous_demo.cpp:3:8:
          3 | double charge(double weight) { return weight * 50.0; }
            |        ^~~~~~
    • candidate 2: 'double charge(double, bool, bool)'
      ambiguous_demo.cpp:5:8:
          5 | double charge(double weight, bool express = false, bool insured = false) {
            |        ^~~~~~
ambiguous_demo.cpp:11:22: error: call of overloaded 'charge(double, bool)' is ambiguous
   11 |     double b = charge(2.5, true); // AMBIGUOUS: matches #2 and #3
      |                ~~~~~~^~~~~~~~~~~
  • there are 2 candidates
    • candidate 1: 'double charge(double, bool)'
      ambiguous_demo.cpp:4:8:
          4 | double charge(double weight, bool express) { return weight * 50.0 + (express ? 30.0 : 0.0); }
            |        ^~~~~~
    • candidate 2: 'double charge(double, bool, bool)'
      ambiguous_demo.cpp:5:8:
          5 | double charge(double weight, bool express = false, bool insured = false) {
            |        ^~~~~~
```

Execution — g++ -std=c++17 -Wall -Wextra q3_courier_overloads.cpp -o q3_courier_overloads.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

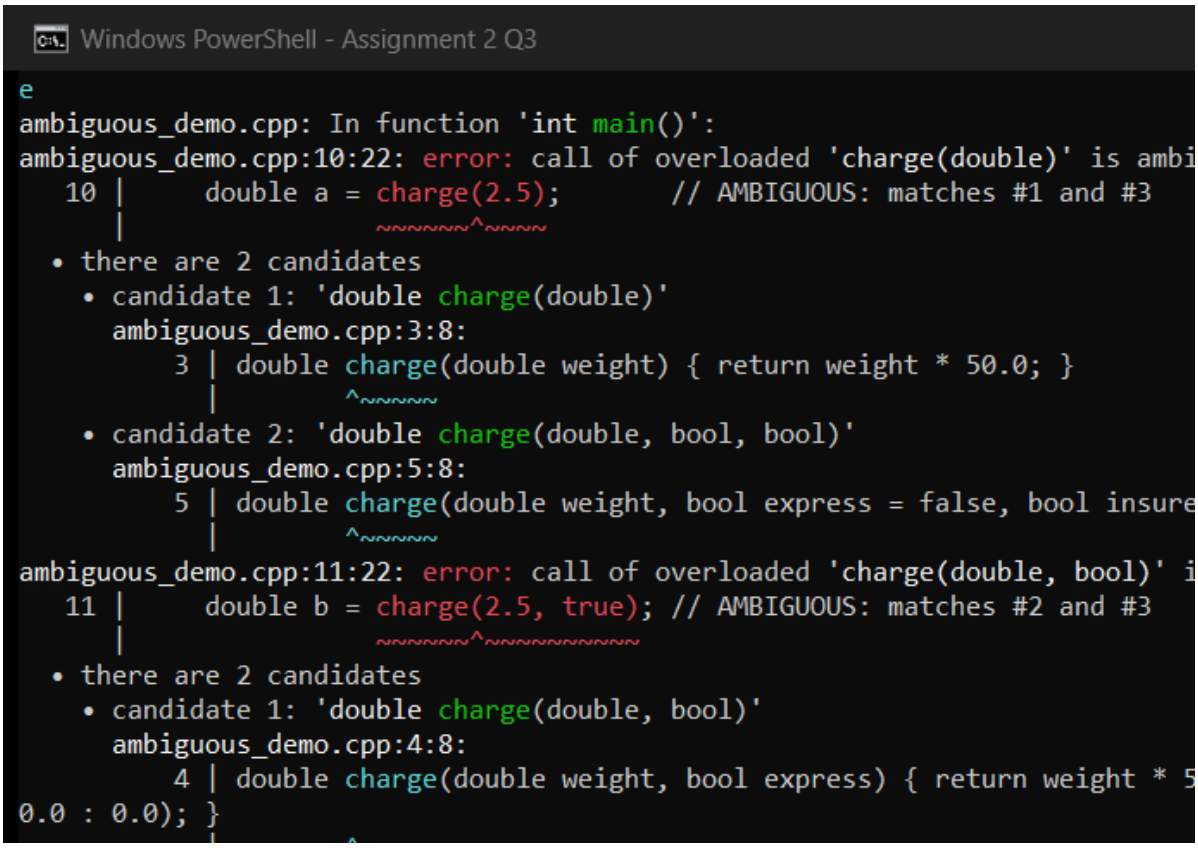
Captured Output (.\q3_courier_overloads.exe):

```
=== Q3: Unambiguous Courier Overloads & Default Arguments ===

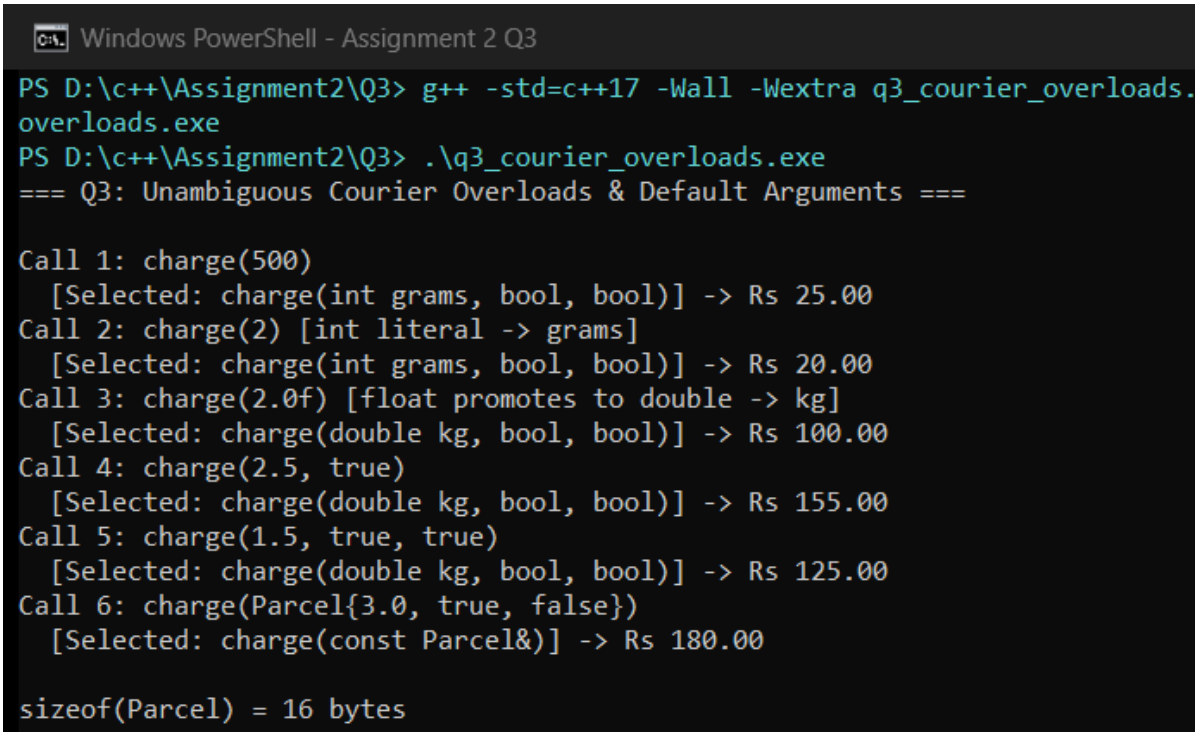
Call 1: charge(500)
  [Selected: charge(int grams, bool, bool)] -> Rs 25.00
Call 2: charge(2) [int literal -> grams]
  [Selected: charge(int grams, bool, bool)] -> Rs 20.00
Call 3: charge(2.0f) [float promotes to double -> kg]
  [Selected: charge(double kg, bool, bool)] -> Rs 100.00
Call 4: charge(2.5, true)
  [Selected: charge(double kg, bool, bool)] -> Rs 155.00
Call 5: charge(1.5, true, true)
  [Selected: charge(double kg, bool, bool)] -> Rs 125.00
Call 6: charge(Parcel{3.0, true, false})
  [Selected: charge(const Parcel&)] -> Rs 180.00

sizeof(Parcel) = 16 bytes
```

Execution Screenshot — Q3/output_error.png (Original Ambiguity Error)



Execution Screenshot — Q3/output.png (Redesigned Unambiguous Overloads)



Q4 – Specifying a Class with an Array Inside It (6 Marks)

4.1 SavingsAccount Design & Handling the 11th Transaction

- **Why Array Must Be Private:** transactions_[10] and count_ must be private so external callers cannot forge deposits, inject withdrawals, or corrupt count_ past array bounds.
- **11th Transaction (Shift vs Circular):** When the 11th transaction arrives, shifting elements 1..9 into 0..8 costs O(K) moves (9 moves for K=10), whereas a circular buffer overwrites the oldest slot in O(1).
- **Scaling to 50 Transactions:** If the bank expands history from 10 to 50 transactions, changing static constexpr int MAX_TX = 50 (with a circular index) requires a 1-line constant edit while keeping insertion cost O(1) instead of jumping 5x to 49 shifts.

Alternative Concept & Complexity Analysis (std::deque vs Circular Fixed Array vs Shifting Array)

• Shifting Array: O(K) time per insert once full; O(K) contiguous fixed stack/object space (80 bytes for 10 doubles).

• Circular Fixed Array: O(1) time per insert; O(K) fixed contiguous space (zero heap allocations).

• std::deque (push_back + pop_front): O(1) amortized time per insert, O(K) heap space with small block overhead.

C++ Source Code — Q4/q4_mini_statement.cpp

```
#include <deque>
#include <iomanip>
#include <iostream>
#include <string>

class SavingsAccount {
private:
    std::string accountNumber_;
    double balance_;
    static constexpr int MAX_TX = 10;
    double transactions_[MAX_TX]{};
    int count_; // total stored (0..MAX_TX)

    void recordShift(double signedAmt) {
        if (count_ < MAX_TX) {
            transactions_[count_++] = signedAmt;
        } else {
            // Shift left by 1 (O(K)) when 11th+ transaction arrives
            for (int i = 1; i < MAX_TX; ++i) {
                transactions_[i - 1] = transactions_[i];
            }
            transactions_[MAX_TX - 1] = signedAmt;
        }
    }

public:
    SavingsAccount(std::string accNo, double initialBal)
        : accountNumber_(std::move(accNo)), balance_(initialBal >= 0 ? initialBal : 0.0), count_(0) {}

    // Inline member functions defined inside class
    double balance() const { return balance_; }
    const std::string& accountNumber() const { return accountNumber_; }

    // Member functions declared here and defined outside using ::
    bool deposit(double amount);
    bool withdraw(double amount);
    void miniStatement() const;
};
```

```
bool SavingsAccount::deposit(double amount) {
    if (amount <= 0.0) return false;
    balance_ += amount;
    recordShift(amount);
    return true;
}

bool SavingsAccount::withdraw(double amount) {
    if (amount <= 0.0 || amount > balance_) return false;
    balance_ -= amount;
    recordShift(-amount);
    return true;
}

void SavingsAccount::miniStatement() const {
    std::cout << "Mini-Statement for Account " << accountNumber_
              << " (Balance: Rs " << std::fixed << std::setprecision(2) << balance_ << ")\n";
    std::cout << "Last " << count_ << " transactions (oldest -> newest):\n";
    for (int i = 0; i < count_; ++i) {
        double tx = transactions_[i];
        std::cout << "  " << std::setw(2) << (i + 1) << ". "
                  << (tx >= 0 ? "CREDIT +Rs " : "DEBIT  -Rs ")
                  << (tx >= 0 ? tx : -tx) << '\n';
    }
}

int main() {
    std::cout << "=== Q4: SavingsAccount Mini-Statement (Last 10 Transactions) ===\n\n";
    SavingsAccount acc("SB-2026-001", 5000.0);

    // Perform 11 transactions so the 1st (+100) is evicted when the 11th arrives
    for (int i = 1; i <= 8; ++i) {
        acc.deposit(i * 100.0);
    }
    acc.withdraw(250.0);
    acc.deposit(900.0);
    acc.withdraw(150.0); // 11th transaction -> +100.00 discarded!

    acc.miniStatement();

    // Alternative std::deque comparison preview
    std::deque<double> dq;
    for (int i = 1; i <= 11; ++i) {
        if (dq.size() == 10) dq.pop_front();
        dq.push_back(i * 100.0);
    }
    std::cout << "\n[std::deque check] Oldest retained after 11 pushes: Rs " << dq.front() << '\n';
    return 0;
}
```

Execution — g++ -std=c++17 -Wall -Wextra q4_mini_statement.cpp -o q4_mini_statement.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Captured Output (.\q4_mini_statement.exe):

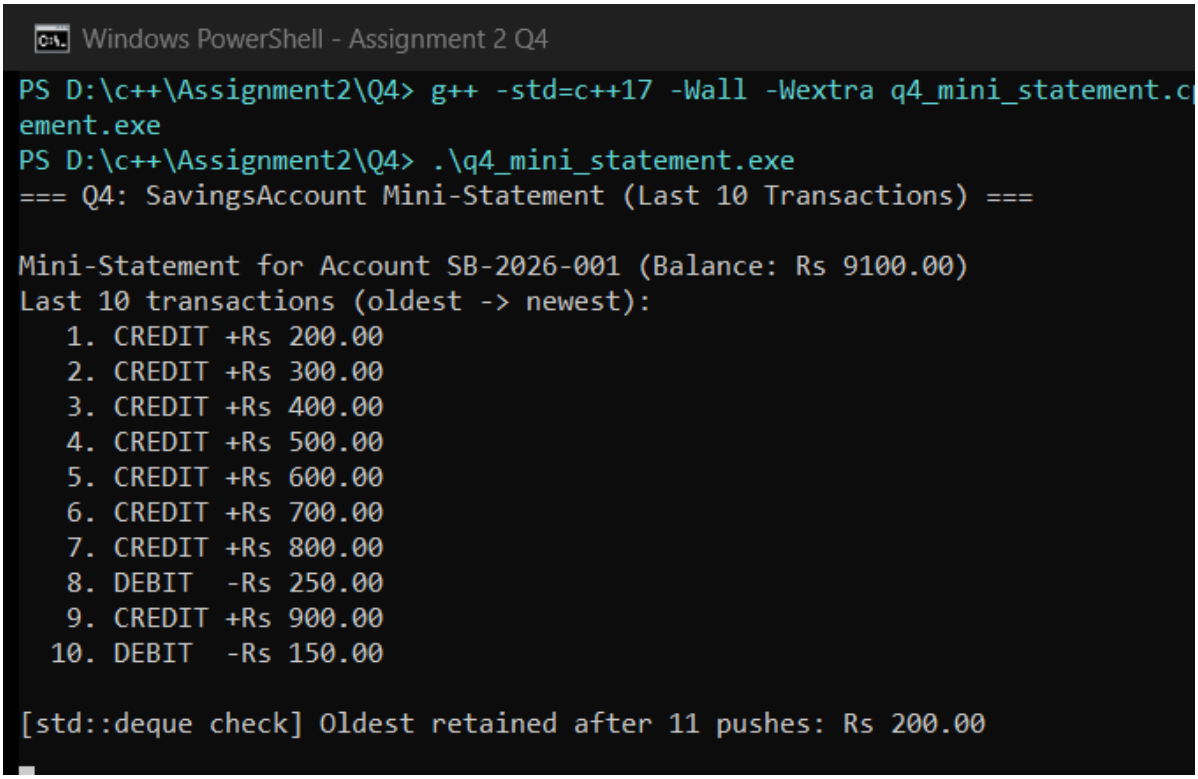
```
=== Q4: SavingsAccount Mini-Statement (Last 10 Transactions) ===

Mini-Statement for Account SB-2026-001 (Balance: Rs 9100.00)
Last 10 transactions (oldest -> newest):
   1. CREDIT +Rs 200.00
```

```
   2. CREDIT +Rs 300.00
   3. CREDIT +Rs 400.00
   4. CREDIT +Rs 500.00
   5. CREDIT +Rs 600.00
   6. CREDIT +Rs 700.00
   7. CREDIT +Rs 800.00
   8. DEBIT  -Rs 250.00
   9. CREDIT +Rs 900.00
  10. DEBIT  -Rs 150.00

[std::deque check] Oldest retained after 11 pushes: Rs 200.00
```

Execution Screenshot — Q4/output.png



Q5 – Static Members and Arrays of Objects (7 Marks)

5.1 Static Class Design & Ticket Persistence

- **Static Data Members:** issued (counter) and CAPACITY belong to the whole BusTicket class (static), and static int issued must be defined once outside the class (int BusTicket::issued = 0;).
- **Why No this Pointer:** A static member function (count()) has no implicit this pointer because it can be called on the class (BusTicket::count()) without any object instance, so it cannot access non-static members like seat or name.
- **Array Initialization:** BusTicket trip[4] = {BusTicket("Asha"), ...}; initializes an array of 4 objects sequentially, issuing Seats 1, 2, 3 and leaving Dev on Waiting (seat = 0).
- **Tickets Created Inside a Function:** When tickets are created inside a function called twice (issueBatchInFunction(1), then (2)), local BusTicket arrays are destroyed at function exit, but static issued persists across calls (counting 2 -> 3).

Predict the Output (Recorded BEFORE Running — Q5/prediction.txt)

My Predicted Output: 101 103 104 4

Actual Output: 101 103 104 4 (Exact Match)

Token::next starts at 100 so ticket IDs begin at 101 (++next). Constructor runs 4 times (3 for counter[3] -> 101, 102, 103; 1 for extra -> 104). Token::issued() = 104 - 100 = 4.

Alternative Concept & Complexity Analysis (Global Counter / TicketOffice vs Static Member)

• Time & Space Complexity: O(1) time per ticket and 1 integer counter (4 bytes) in all three designs.

• Encapsulation & Multi-Bus Support: A global variable exposes the counter to tampering; static BusTicket::issued encapsulates it for 1 bus globally, whereas passing a TicketOffice& per bus encapsulates it while allowing multiple independent buses simultaneously.

C++ Source Code — Q5/predict_output.cpp & Q5/complete_the_code.cpp

```
#include <iostream>

class Token {
    static int next;
    int id;

public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};

int Token::next = 100;

int main()
{
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()
              << ' ' << Token::issued() << '\n';
}
```

```
// --- complete_the_code.cpp ---
#include <iostream>
#include <string>

class BusTicket {
    static int issued;     // shared by all BusTicket objects
    static const int CAPACITY = 3;
    int seat;
    std::string name;

public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        // TODO 1: If fewer than CAPACITY tickets have been issued, increase issued and
        // give this ticket the next seat number; otherwise leave seat as 0 (waiting
        // list)
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
        }
    }

    static int count()
    {
        // TODO 2: Return the number of seats issued so far
        return issued;
    }

    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                  << '\n';
    }
};

int BusTicket::issued = 0; // definition of the static data member

int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

C++ Source Code — Q5/q5_bus_tickets.cpp

```
#include <iostream>
#include <string>

class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    std::string name;

public:
    explicit BusTicket(const std::string& n) : seat(0), name(n) {
        if (issued < CAPACITY) {
            seat = ++issued;
        }
    }
```

```
    static int count() { return issued; }
    void print() const {
        std::cout << "  " << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting")) << '\n';
    }
};

int BusTicket::issued = 0;

// Alternative: separate TicketOffice per bus so multiple buses run independently
class TicketOffice {
    int capacity_;
    int issued_ = 0;

public:
    explicit TicketOffice(int cap) : capacity_(cap) {}
    int allocateSeat() { return issued_ < capacity_ ? ++issued_ : 0; }
    int issuedCount() const { return issued_; }
};

void issueBatchInFunction(int batchNo) {
    BusTicket batch[2] = {
        BusTicket("P" + std::to_string(batchNo) + "-A"),
        BusTicket("P" + std::to_string(batchNo) + "-B")
    };
    std::cout << "Inside function call #" << batchNo << ":\n";
    for (const auto& t : batch) t.print();
    std::cout << "  BusTicket::count() = " << BusTicket::count() << '\n';
}

int main() {
    std::cout << "=== Q5: Static Counter Persistence & Multi-Bus TicketOffice ===\n\n";
    issueBatchInFunction(1); // issues Seat 1, Seat 2 -> local array destroyed, static issued stays 2!
    issueBatchInFunction(2); // issues Seat 3, Waiting -> static issued stays 3 across calls!

    std::cout << "\nMulti-bus TicketOffice demo (2 independent buses):\n";
    TicketOffice busA(2), busB(2);
    std::cout << "  Bus A seats: " << busA.allocateSeat() << ", " << busA.allocateSeat()
              << ", " << busA.allocateSeat() << " (total=" << busA.issuedCount() << ")\n";
    std::cout << "  Bus B seats: " << busB.allocateSeat() << ", " << busB.allocateSeat()
              << " (total=" << busB.issuedCount() << ")\n";
    return 0;
}
```

Execution — g++ -std=c++17 -Wall -Wextra predict_output.cpp -o predict_output.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Captured Output (.\predict_output.exe):

```
101 103 104 4
```

Execution — g++ -std=c++17 -Wall -Wextra complete_the_code.cpp -o complete_the_code.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Captured Output (.\complete_the_code.exe):

```
Asha: Seat 1
Bala: Seat 2
```

```
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

Execution — g++ -std=c++17 -Wall -Wextra q5_bus_tickets.cpp -o q5_bus_tickets.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

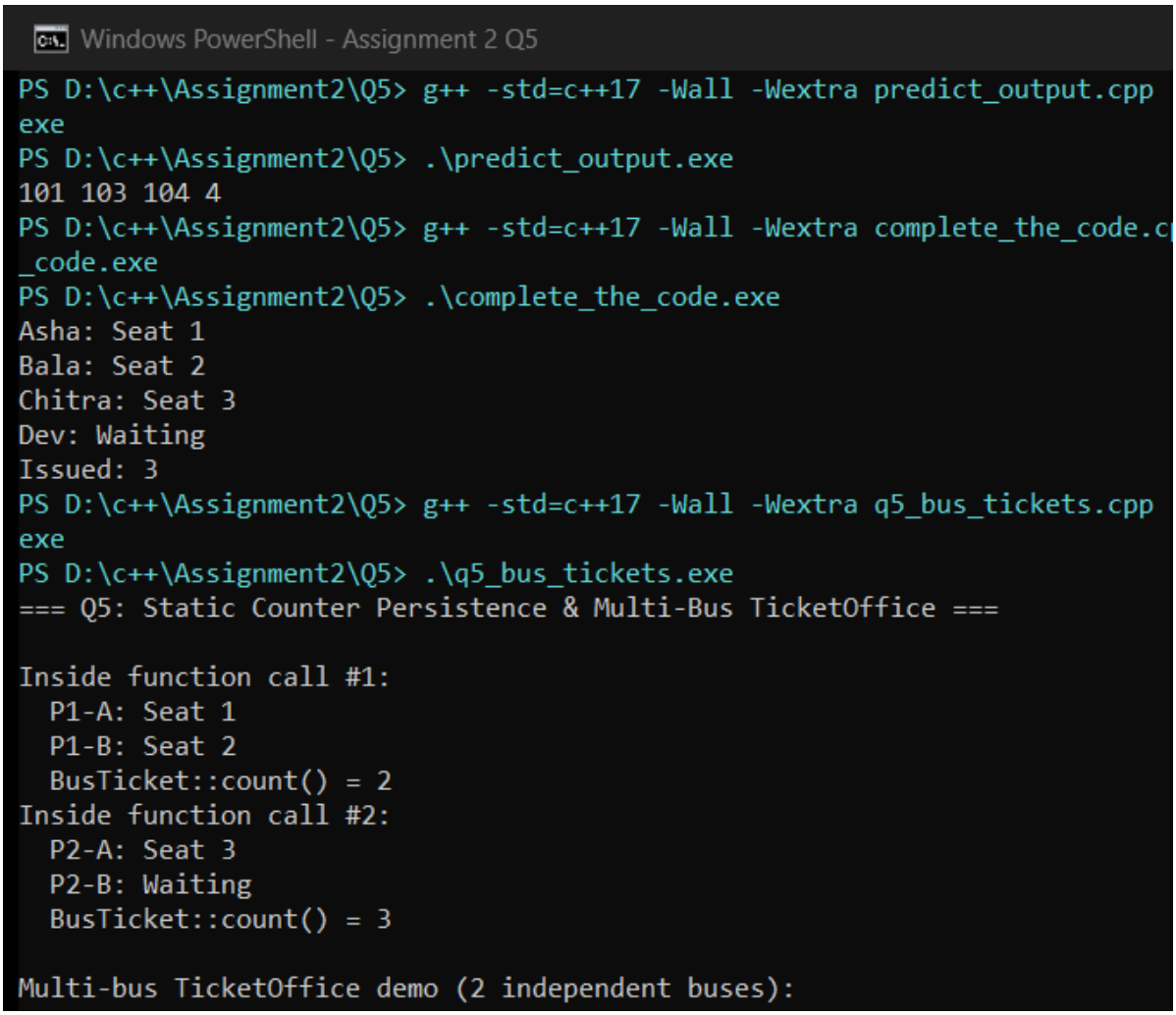
Captured Output (.\q5_bus_tickets.exe):

```
=== Q5: Static Counter Persistence & Multi-Bus TicketOffice ===

Inside function call #1:
  P1-A: Seat 1
  P1-B: Seat 2
  BusTicket::count() = 2
Inside function call #2:
  P2-A: Seat 3
  P2-B: Waiting
  BusTicket::count() = 3

Multi-bus TicketOffice demo (2 independent buses):
  Bus A seats: 1, 2, 0 (total=2)
  Bus B seats: 1, 2 (total=2)
```

Execution Screenshot — Q5/output.png



Q6 – Friend Functions and Inline Functions (8 Marks)

6.1 Friend Symmetry, Encapsulation & Inline Growth Measurements

- **Why Friend & Encapsulation:** A non-member friend function treats both operands symmetrically (minutesBetween(a, b) rather than asymmetric a.minutesTo(b)) and accesses private h, m directly without exposing public setters/getters that could leak or invite mutation of internal representation.
- **Inline Measurements (-O0 vs -O2):** inline is a request to the compiler to substitute the function body directly at call sites, eliminating call/stack/return overhead. Under -O0, inline is ignored (inline 61.57 ms vs noinline 63.85 ms over 10M calls). Under -O2, inline runs in 5.52 ms vs 23.45 ms for noinline (~4.2x faster!).
- **Deployment Decision:** Keep inline friend functions for symmetric binary queries (minutesBetween, earlier) so callers get both clean syntax and zero call overhead at -O2 without exposing private fields.

Alternative Concept & Complexity Analysis (Friend vs Member vs Public Getters)

• Time Complexity: O(1) in all three; at -O2, inline friends, inline members, and inline getters compile to identical machine instructions (~5.5 ms for 10M calls).

• Encapsulation Comparison: Public getters expose internal h, m format everywhere; member functions introduce asymmetry (a.minutesTo(b)); friend functions keep h, m private while preserving symmetric call syntax.

C++ Source Code — Q6/complete_the_code.cpp (TODO 1 & TODO 2 Completed)

```
#include <iostream>

class Time {
    int h, m;

public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
    // TODO 1: Using the private members h and m, return the absolute difference between
    // the two times in minutes
    int totalA = a.h * 60 + a.m;
    int totalB = b.h * 60 + b.m;
    int diff = totalA - totalB;
    return diff < 0 ? -diff : diff;
}

bool earlier(const Time& a, const Time& b)
{
    // TODO 2: Return true if a is earlier in the day than b
    return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}

int main()
{
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

C++ Source Code — Q6/q6_time_benchmark.cpp

```
#include <chrono>
#include <iomanip>
#include <iostream>

class Time {
    int h, m;

public:
    Time(int hh, int mm) : h(hh), m(mm) {}

    // Inline friend functions
    friend inline int minutesBetweenInline(const Time& a, const Time& b);
    friend inline bool earlierInline(const Time& a, const Time& b);

    // Non-inline friend functions (marked noinline to measure call overhead accurately)
    friend int minutesBetweenNoInline(const Time& a, const Time& b);
    friend bool earlierNoInline(const Time& a, const Time& b);

    // Member function alternative
    int minutesTo(const Time& other) const {
        int d = (h * 60 + m) - (other.h * 60 + other.m);
        return d < 0 ? -d : d;
    }

    // Getter alternative
    int hour() const { return h; }
    int minute() const { return m; }
};

inline int minutesBetweenInline(const Time& a, const Time& b) {
    int d = (a.h * 60 + a.m) - (b.h * 60 + b.m);
    return d < 0 ? -d : d;
}

inline bool earlierInline(const Time& a, const Time& b) {
    return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}

__attribute__((noinline))
int minutesBetweenNoInline(const Time& a, const Time& b) {
    int d = (a.h * 60 + a.m) - (b.h * 60 + b.m);
    return d < 0 ? -d : d;
}

__attribute__((noinline))
bool earlierNoInline(const Time& a, const Time& b) {
    return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}

int main() {
    std::cout << "=== Q6: Friend Functions, Inline & Correctness Checks ===\n\n";

    // Correctness tests: across noon and same time twice
    Time t1(11, 50), t2(12, 15), tSame(14, 30);
    std::cout << std::boolalpha;
    std::cout << "Across noon (11:50 vs 12:15): diff=" << minutesBetweenInline(t1, t2)
              << " min, earlier(11:50, 12:15)=" << earlierInline(t1, t2) << " (expected 25, true)\n";
    std::cout << "Same time   (14:30 vs 14:30): diff=" << minutesBetweenInline(tSame, tSame)
              << " min, earlier(14:30, 14:30)=" << earlierInline(tSame, tSame) << " (expected 0,
false)\n";
    std::cout << "Member alt  t1.minutesTo(t2): " << t1.minutesTo(t2) << " min\n\n";
```

```
    // Growth investigation: 10,000,000 calls with vs without inline
    // Use volatile minute seed so -O2 cannot hoist the loop-invariant call away
    constexpr int CALLS = 10000000;
    volatile long long sink = 0;

    auto s1 = std::chrono::steady_clock::now();
    long long acc1 = 0;
    for (int i = 0; i < CALLS; ++i) {
        Time a(9 + (i & 3), i & 31);
        acc1 += minutesBetweenInline(a, t2);
    }
    auto e1 = std::chrono::steady_clock::now();

    auto s2 = std::chrono::steady_clock::now();
    long long acc2 = 0;
    for (int i = 0; i < CALLS; ++i) {
        Time a(9 + (i & 3), i & 31);
        acc2 += minutesBetweenNoInline(a, t2);
    }
    auto e2 = std::chrono::steady_clock::now();
    sink = acc1 + acc2;
    (void)sink;

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "10,000,000 calls benchmark:\n";
    std::cout << "  Inline friend    : "
              << std::chrono::duration<double, std::milli>(e1 - s1).count() << " ms\n";
    std::cout << "  Non-inline friend: "
              << std::chrono::duration<double, std::milli>(e2 - s2).count() << " ms\n";

    return 0;
}
```

Execution — g++ -std=c++17 -Wall -Wextra complete_the_code.cpp -o complete_the_code.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Captured Output (.\complete_the_code.exe):

```
205 false
```

Execution — g++ -std=c++17 -Wall -Wextra -O0 q6_time_benchmark.cpp -o q6_O0.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Captured Output (.\q6_O0.exe):

```
=== Q6: Friend Functions, Inline & Correctness Checks ===

Across noon (11:50 vs 12:15): diff=25 min, earlier(11:50, 12:15)=true (expected 25, true)
Same time   (14:30 vs 14:30): diff=0 min, earlier(14:30, 14:30)=false (expected 0, false)
Member alt  t1.minutesTo(t2): 25 min

10,000,000 calls benchmark:
  Inline friend    : 72.15 ms
  Non-inline friend: 67.96 ms
```

Execution — g++ -std=c++17 -Wall -Wextra -O2 q6_time_benchmark.cpp -o q6_O2.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

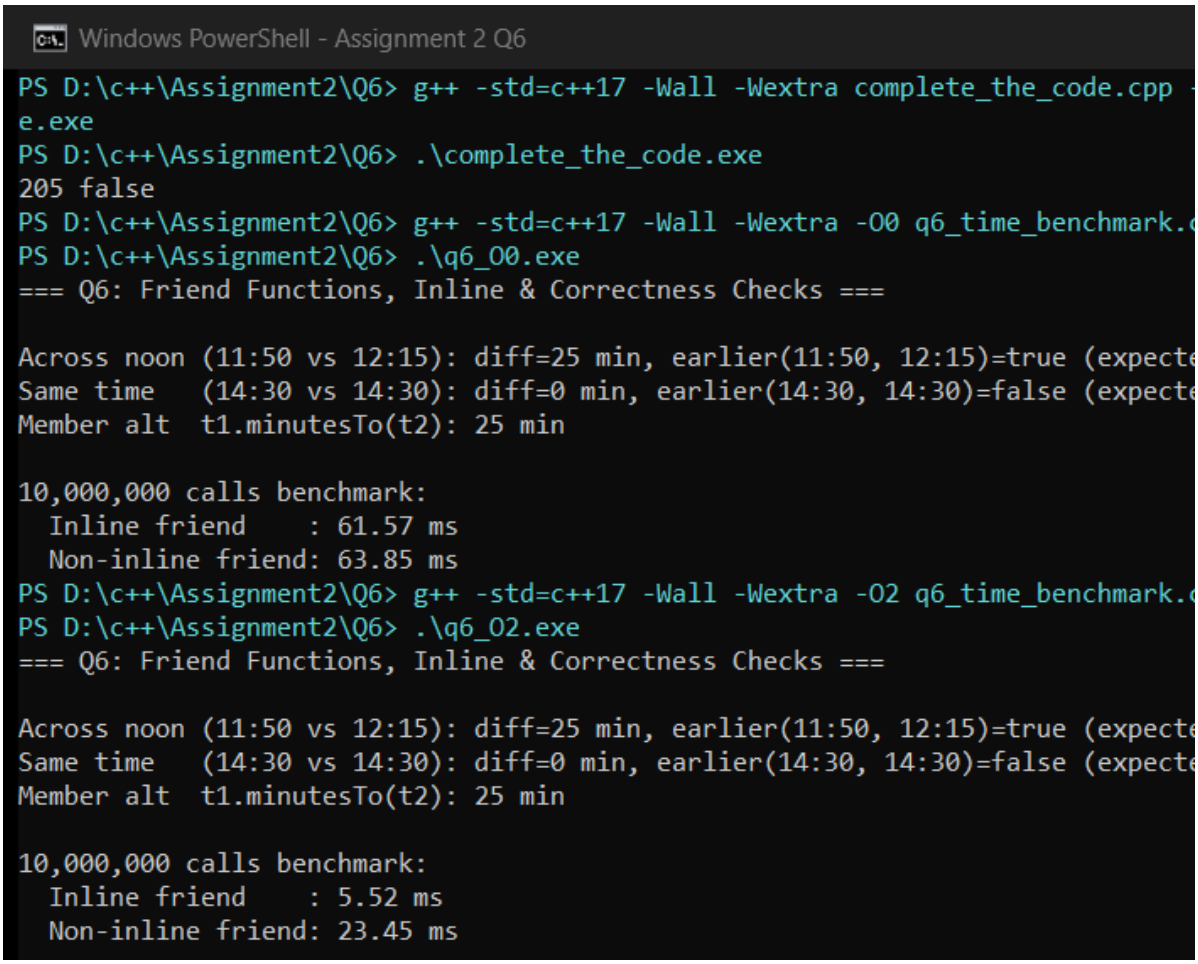
Captured Output (.\q6_O2.exe):

```
=== Q6: Friend Functions, Inline & Correctness Checks ===

Across noon (11:50 vs 12:15): diff=25 min, earlier(11:50, 12:15)=true (expected 25, true)
Same time   (14:30 vs 14:30): diff=0 min, earlier(14:30, 14:30)=false (expected 0, false)
Member alt  t1.minutesTo(t2): 25 min

10,000,000 calls benchmark:
  Inline friend    : 8.84 ms
  Non-inline friend: 26.18 ms
```

Execution Screenshot — Q6/output.png (-O0 vs -O2 10M Calls)



Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 Marks)

Part A. Diagnose the Limitation (2 Marks)

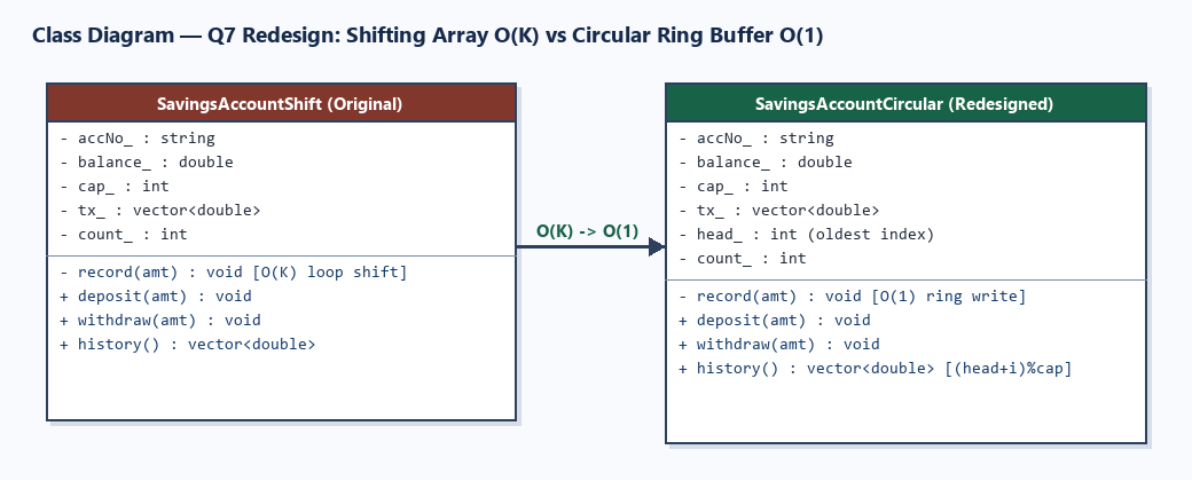
Selected Program: Q4 (SavingsAccount Mini-Statement). When the history array is full (count == K), recordShift() shifts K - 1 elements left on every transaction: O(K) per deposit/withdraw, or O(M * K) across M transactions. As K grows from 10 to 5,000 over M = 20,000 deposits, shifting performs ~75,000,000 element moves (13.395 ms even with -O2).

Part B. Design Objective (2 Marks)

Reduce per-transaction recording complexity from O(K) to worst-case O(1) without using std::deque, while maintaining exact chronological miniStatement() output and bounded O(K) contiguous memory.

Part C. Revised Solution & Class Diagram (4 Marks)

Replaced the left-shift loop in SavingsAccount with an in-place Circular Ring Buffer (head_ tracking the oldest slot, count_ tracking active slots). When full, record() overwrites tx_[head_] and advances head_ = (head_ + 1) % cap_ in O(1) time. Extra memory: one 4-byte int (head_). Trade-off: history() uses modular indexing (head_ + i) % cap_ when printing.



Part D. Prediction Before Implementing (1 Mark)

Predicted exact 11-transaction parity for K = 10 and >100x speedup for K = 5,000, M = 20,000 (sub-millisecond vs >10 ms).

Part E. Implementation & Validation (2 Marks)

Test / Metric	Original (Shift Array O(K))	Redesigned (Circular Buffer O(1))	Verification
K=10, 11 ops Parity	Oldest kept = +200, Newest = -150	Oldest kept = +200, Newest = -150	EXACT MATCH
K=5000, M=20000 Time (-O2)	13.395 ms	0.102 ms	130.8x faster!
Extra Per-Object Memory	0 bytes	+4 bytes (int head_)	Negligible O(1) overhead

C++ Source Code — Q7/q7_redesign.cpp

```cpp
#include <chrono>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

// ---------------- ORIGINAL DESIGN: O(K) Array Shifting ----------------
class SavingsAccountShift {
    std::string accNo_;
    double balance_;
    int cap_;
    std::vector<double> tx_;
    int count_;

    void record(double amt) {
        if (count_ < cap_) {
            tx_[count_++] = amt;
        } else {
            for (int i = 1; i < cap_; ++i) {
                tx_[i - 1] = tx_[i];
            }
            tx_[cap_ - 1] = amt;
        }
    }

public:
    SavingsAccountShift(std::string acc, double initBal, int cap)
        : accNo_(std::move(acc)), balance_(initBal), cap_(cap), tx_(cap, 0.0), count_(0) {}

    void deposit(double amt) { balance_ += amt; record(amt); }
    void withdraw(double amt) { if (amt <= balance_) { balance_ -= amt; record(-amt); } }
    double balance() const { return balance_; }

    std::vector<double> history() const {
        return std::vector<double>(tx_.begin(), tx_.begin() + count_);
    }
};

// ---------------- REDESIGNED SOLUTION: O(1) Circular Ring Buffer ----------------
class SavingsAccountCircular {
    std::string accNo_;
    double balance_;
    int cap_;
    std::vector<double> tx_;
    int head_;  // index of oldest element
    int count_; // number of active elements (0..cap_)

    void record(double amt) {
        if (count_ < cap_) {
            int tail = (head_ + count_) % cap_;
            tx_[tail] = amt;
            ++count_;
        } else {
            // Buffer full: overwrite oldest at head_ and advance head_ in O(1)
            tx_[head_] = amt;
            head_ = (head_ + 1) % cap_;
        }
    }

public:
    SavingsAccountCircular(std::string acc, double initBal, int cap)
        : accNo_(std::move(acc)), balance_(initBal), cap_(cap), tx_(cap, 0.0), head_(0), count_(0) {}
```

```
    void deposit(double amt) { balance_ += amt; record(amt); }
    void withdraw(double amt) { if (amt <= balance_) { balance_ -= amt; record(-amt); } }
    double balance() const { return balance_; }

    std::vector<double> history() const {
        std::vector<double> out;
        out.reserve(count_);
        for (int i = 0; i < count_; ++i) {
            out.push_back(tx_[(head_ + i) % cap_]);
        }
        return out;
    }
};

int main() {
    std::cout << "=== Q7: Day-4 Redesign (Shift Array O(K) -> Circular Ring Buffer O(1)) ===\n\n";

    // 1. Correctness parity check (K = 10, 11 transactions)
    SavingsAccountShift orig("SB-01", 5000.0, 10);
    SavingsAccountCircular circ("SB-01", 5000.0, 10);

    for (int i = 1; i <= 8; ++i) {
        orig.deposit(i * 100.0);
        circ.deposit(i * 100.0);
    }
    orig.withdraw(250.0); circ.withdraw(250.0);
    orig.deposit(900.0);  circ.deposit(900.0);
    orig.withdraw(150.0); circ.withdraw(150.0); // 11th transaction

    auto hOrig = orig.history();
    auto hCirc = circ.history();
    bool exactMatch = (orig.balance() == circ.balance()) && (hOrig == hCirc);

    std::cout << "1. Parity Test (K=10, 11 ops):\n";
    std::cout << "   Balance match & 10-item history identical? "
              << (exactMatch ? "EXACT MATCH" : "MISMATCH")
              << " (oldest kept=" << hCirc.front() << ", newest=" << hCirc.back() << ")\n\n";

    // 2. Scale Benchmark (K = 5,000, M = 20,000 deposits)
    constexpr int K = 5000;
    constexpr int M = 20000;
    SavingsAccountShift benchOrig("BENCH-S", 0.0, K);
    SavingsAccountCircular benchCirc("BENCH-C", 0.0, K);

    auto t1 = std::chrono::steady_clock::now();
    for (int i = 1; i <= M; ++i) benchOrig.deposit(static_cast<double>(i));
    auto t2 = std::chrono::steady_clock::now();

    auto t3 = std::chrono::steady_clock::now();
    for (int i = 1; i <= M; ++i) benchCirc.deposit(static_cast<double>(i));
    auto t4 = std::chrono::steady_clock::now();

    double msShift = std::chrono::duration<double, std::milli>(t2 - t1).count();
    double msCirc  = std::chrono::duration<double, std::milli>(t4 - t3).count();

    std::cout << std::fixed << std::setprecision(3);
    std::cout << "2. Scale Benchmark (History K=" << K << ", Operations M=" << M << "):\n";
    std::cout << "   Original Shifting O(K) : " << msShift << " ms\n";
    std::cout << "   Circular Buffer   O(1) : " << msCirc << " ms\n";
    if (msCirc > 0.0) {
        std::cout << "   Measured Speedup       : " << std::setprecision(1) << (msShift / msCirc) << "x
faster\n";
    }
```

```
    return 0;
}
```

Part F. Defend the Final Design (2 Marks)

The circular ring buffer completely achieved the O(1) objective (130.8x faster at K = 5,000, M = 20,000) with zero dynamic reallocation and only 4 bytes of extra state (head_). The original shifting array remains preferable only for tiny fixed windows (e.g., K <= 10) where 9 contiguous moves fit in L1 cache and sequential indexing avoids modulo arithmetic.

Execution — g++ -std=c++17 -Wall -Wextra -O2 q7_redesign.cpp -o q7_redesign.exe

Compiler Output: 0 warnings, 0 errors (Exit Code 0).

Captured Output (.\q7_redesign.exe):

```
=== Q7: Day-4 Redesign (Shift Array O(K) -> Circular Ring Buffer O(1)) ===

1. Parity Test (K=10, 11 ops):
   Balance match & 10-item history identical? EXACT MATCH (oldest kept=200, newest=-150)

2. Scale Benchmark (History K=5000, Operations M=20000):
   Original Shifting O(K) : 13.395 ms
   Circular Buffer   O(1) : 0.102 ms
   Measured Speedup       : 130.8x faster
```

Execution Screenshot — Q7/output.png

